

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО
МАТЕРИАЛА ***Лекция – 13-14***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Лекция – 13-14

Цели и задачи и виды тестирования. Стандарты качества программной документации. Меры и метрики

ОРГАНИЗАЦИЯ ПРОЦЕССА ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

В этой главе излагаются вопросы, связанные с проведением тестирования на всех этапах конструирования программной системы. Классический процесс тестирования обеспечивает проверку результатов, полученных на каждом этапе разработки. Как правило, он начинается с тестирования в малом, когда проверяются программные модули, продолжается при проверке объединения модулей в систему и завершается тестированием в большом, при котором проверяются соответствие программного продукта требованиям заказчика и его взаимодействие с другими компонентами компьютерной системы. Данная глава последовательно описывает содержание каждого шага тестирования. Здесь же рассматривается организация отладки ПО, которая проводится для устранения выявленных при тестировании ошибок.

Методика тестирования программных систем

Процесс тестирования объединяет различные способы тестирования в спланированную последовательность шагов, которые приводят к успешному построению программной системы (ПС) Методика тестирования ПС может быть представлена в виде разворачивающейся спирали (рис. 8.1).

В начале осуществляется *тестирование элементов (модулей),* проверяющее результаты этапа *кодирования* ПС. На втором шаге выполняется *тестирование интеграции,* ориентированное на выявление ошибок этапа *проектирования* ПС. На третьем обороте спирали производится *тестирование правильности,* проверяющее корректность этапа *анализа требований* к ПС. На заключительном витке спирали проводится *системное тестирование,* выявляющее дефекты этапа *системного анализа* ПС.

Охарактеризуем каждый шаг процесса тестирования.

1. *Тестирование элементов.* Цель — индивидуальная проверка каждого модуля. Используются способы тестирования «белого ящика».



Рис. 8.1. Спираль процесса тестирования ПС

2. *Тестирование интеграции.* Цель — тестирование сборки модулей в программную систему. В основном применяют способы тестирования «черного ящика».

3. *Тестирование правильности.* Цель — проверить реализацию в программной системе всех функциональных и поведенческих требований, а также требования эффективности. Используются исключительно способы тестирования «черного ящика».

4. *Системное тестирование.* Цель — проверка правильности объединения и взаимодействия всех элементов компьютерной системы, реализации всех системных функций.

Организация процесса тестирования в виде эволюционной разворачивающейся спирали обеспечивает максимальную эффективность поиска ошибок. Однако возникает вопрос — когда заканчивать тестирование?

Ответ практика обычно основан на статистическом критерии: «Можно с 95%-ной уверенностью сказать, что провели достаточное тестирование, если вероятность безотказной работы ЦП с программным изделием в течение 1000 часов составляет по меньшей мере 0,995».

Научный подход при ответе на этот вопрос состоит в применении математической модели отказов.

Тестирование элементов

Объектом тестирования элементов является наименьшая единица проектирования ПС — модуль. Для обнаружения ошибок в рамках модуля тестируются его важнейшие управляющие пути. Относительная сложность тестов и ошибок определяется как результат ограничений области тестирования элементов. Принцип тестирования — «белый ящик», шаг может выполняться для набора модулей параллельно.

Тестированию подвергаются:

- интерфейс модуля;
- внутренние структуры данных;
- независимые пути;
- пути обработки ошибок;
- граничные условия.

Интерфейс модуля тестируется для проверки правильности ввода-вывода тестовой информации. Если нет уверенности в правильном вводе-выводе данных, нет смысла проводить другие тесты.

Исследование внутренних структур данных гарантирует целостность сохраняемых данных.

Тестирование независимых путей гарантирует однократное выполнение всех операторов модуля. При тестировании путей выполнения обнаруживаются следующие категории ошибок: ошибочные вычисления, некорректные сравнения, неправильный поток управления.

Наиболее общими ошибками вычислений являются:

1) неправильный или непонятый приоритет арифметических операций;
2) смешанная форма операций;
3) некорректная инициализация;
4) несогласованность в представлении точности;
5) некорректное символическое представление выражений.

Источниками ошибок сравнения и неправильных потоков управления являются:

1) сравнение различных типов данных;
2) некорректные логические операции и приоритетность;
3) ожидание эквивалентности в условиях, когда ошибки точности делают эквивалентность невозможной;
4) некорректное сравнение переменных;
5) неправильное прекращение цикла;

6) отказ в выходе при отклонении итерации;
7) неправильное изменение переменных цикла.

Обычно при проектировании модуля предвидят некоторые ошибочные условия. Для защиты от ошибочных условий в модуль вводят пути обработки ошибок. Такие пути тоже должны тестироваться. Тестирование путей обработки ошибок можно ориентировать на следующие ситуации:

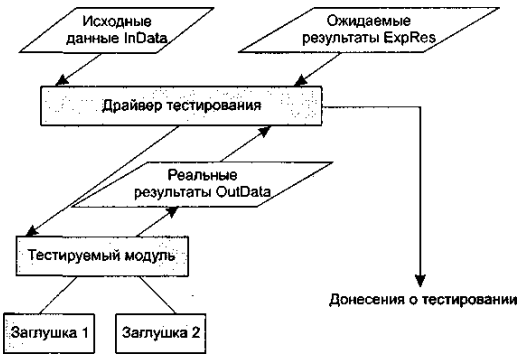
1) донесение об ошибке невразумительно;
2) текст донесения не соответствует, обнаруженной ошибке;
3) вмешательство системных средств регистрации аварии произошло до обработки ошибки в модуле;
4) обработка исключительного условия некорректна;
5) описание ошибки не позволяет определить ее причину.

И, наконец, перейдем к граничному тестированию. Модули часто отказывают на «границах». Это означает, что ошибки часто происходят:

1) при обработке *n*-го элемента *n*-элементного массива;
2) при выполнении *m*-й итерации цикла с *m* проходами;
3) при появлении минимального (максимального) значения.

Тестовые варианты, ориентированные на данные ситуации, имеют высокую вероятность обнаружения ошибок.

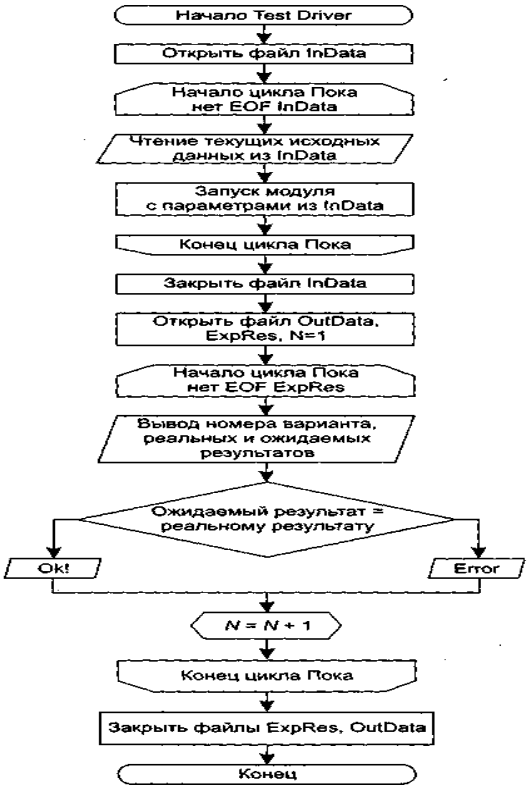
Тестирование элементов обычно рассматривается как дополнение к этапу кодирования. Оно начинается после разработки текста модуля. Так как модуль не является



автономной системой, то для реализации тестирования требуются дополнительные средства, представленные на рис. 8.2.

Рис. 8.2. Программная среда для тестирования модуля

Дополнительными средствами являются драйвер тестирования и заглушки. Драйвер — управляющая программа, которая принимает исходные данные (InData) и ожидаемые результаты (ExpRes) тестовых вариантов, запускает в работу тестируемый модуль,



получает из модуля реальные результаты (OutData) и формирует донесения о тестировании. Алгоритм работы тестового драйвера приведен на рис. 8.3.

Рис. 8.3. Алгоритм работы драйвера тестирования

Заглушки замещают модули, которые вызываются тестируемым модулем. Заглушка, или «фиктивная подпрограмма», реализует интерфейс подчиненного модуля, может выполнять минимальную обработку данных, имитирует прием и возврат данных.

Создание драйвера и заглушек подразумевает дополнительные затраты, так как они не поставляются с конечным программным продуктом.

Если эти средства просты, то дополнительные затраты невелики. Увы, многие модули не могут быть адекватно протестированы с помощью простых дополнительных средств. В этих случаях полное тестирование может быть отложено до шага тестирования интеграции (где драйверы или заглушки также используются).

Тестирование элемента просто осуществить, если модуль имеет высокую связность. При реализации модулем только одной функции количество тестовых вариантов уменьшается, а ошибки легко предсказываются и обнаруживаются.

Тестирование интеграции

Тестирование интеграции поддерживает сборку цельной программной системы.

Цель сборки и тестирования интеграции: взять модули, протестированные как элементы, и построить программную структуру, требуемую проектом [3].

Тесты проводятся для обнаружения ошибок интерфейса. Перечислим некоторые категории ошибок интерфейса:

- потеря данных при прохождении через интерфейс;
- отсутствие в модуле необходимой ссылки;

- неблагоприятное влияние одного модуля на другой;
- подфункции при объединении не образуют требуемую главную функцию;
- отдельные (допустимые) неточности при интеграции выходят за допустимый уровень;
- проблемы при работе с глобальными структурами данных.

Существует два варианта тестирования, поддерживающих процесс интеграции: нисходящее тестирование и восходящеетестирование. Рассмотрим каждый из них.

Нисходящее тестирование интеграции

В данном подходе модули объединяются движением сверху вниз по управляющей иерархии, начиная от главного управляющего модуля. Подчиненные модули добавляются в структуру или в результате поиска в глубину, или в результате поиска в ширину.

Рассмотрим пример (рис. 8.4). Интеграция поиском в глубину будет подключать все модули, находящиеся на главном управляющем пути структуры (по вертикали). Выбор главного управляющего пути отчасти произволен и зависит от характеристик, определяемых приложением. Например, при выборе левого пути прежде всего будут подключены модули Ml, M2, M5. Следующим подключается модуль M8 или Мб (если это необходимо для правильного функционирования M2). Затем строится центральный или правый управляющий путь.

При интеграции поиском в ширину структура последовательно проходится по уровням-горизонталям. На каждом уровне подключаются модули, непосредственно подчиненные управляющему модулю — начальнику. В этом случае прежде всего подключаются модули M2, M3, M4. На следующем уровне — модули M5, Мб и т. д.

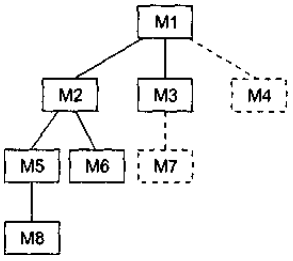


Рис. 8.4. Нисходящая интеграция системы

Опишем возможные шаги процесса нисходящей интеграции.

1. Главный управляющий модуль (находится на вершине иерархии) используется как тестовый драйвер. Все непосредственно подчиненные ему модули временно замещаются заглушками.
2. Одна из заглушек заменяется реальным модулем. Модуль выбирается поиском в ширину или в глубину.
3. После подключения каждого модуля (и установки на нем заглушек) проводится набор тестов, проверяющих полученную структуру.
4. Если в модуле-драйвере уже нет заглушек, производится смена модуля-

драйвера (поиском в ширину или в глубину).

5. Выполняется возврат на шаг 2 (до тех пор, пока не будет построена целая структура).

Достоинство нисходящей интеграции: ошибки в главной, управляющей части системы выявляются в первую очередь.

Недостаток: трудности в ситуациях, когда для полного тестирования на верхних уровнях нужны результаты обработки с нижних уровней иерархии.

Существуют 3 возможности борьбы с этим недостатком:

1) откладывать некоторые тесты до замещения заглушек модулями;
2) разрабатывать заглушки, частично выполняющие функции модулей;
3) подключать модули движением снизу вверх.

Первая возможность вызывает сложности в оценке результатов тестирования.

Для реализации второй возможности выбирается одна из следующих категорий заглушек:

- заглушка A — отображает трассируемое сообщение;
- заглушка B — отображает проходящий параметр;
- заглушка C — возвращает величину из таблицы;
- заглушка D — выполняет табличный поиск по ключу (входному параметру) и возвращает связанный с ним выходной параметр.

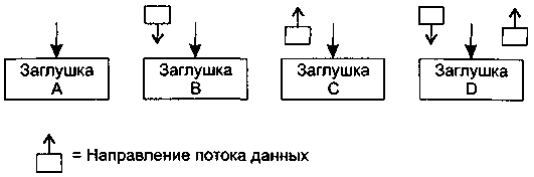


Рис. 8.5. Категории заглушек

Категории заглушек представлены на рис. 8.5.

Очевидно, что заглушка A наиболее проста, а заглушка D наиболее сложна в реализации.

Этот подход работоспособен, но может привести к существенным затратам, так как заглушки становятся все более сложными.

Третью возможность обсудим отдельно.

Восходящее тестирование интеграции

При восходящем тестировании интеграции сборка и тестирование системы начинаются с модулей-атомов, располагаемых на нижних уровнях иерархии. Модули подключаются движением снизу вверх. Подчиненные модули всегда доступны, и нет необходимости в заглушках.

Рассмотрим шаги методики восходящей интеграции.

1. Модули нижнего уровня объединяются в кластеры (группы, блоки), выполняющие определенную программную подфункцию.
2. Для координации вводов-выводов тестового варианта пишется драйвер, управляющий тестированием кластеров.
3. Тестируется кластер.
4. Драйверы удаляются, а кластеры объединяются в структуру движением

вверх. Пример восходящей интеграции системы приведен на рис. 8.6.

Модули объединяются в кластеры 1,2,3. Каждый кластер тестируется драйвером. Модули в кластерах 1 и 2 подчинены модулю Ma, поэтому драйверы D1 и D2 удаляются и кластеры подключают прямо к Ma. Аналогично драйвер D3 удаляется перед подключением кластера 3 к модулю Mb. В последнюю очередь к модулю Mc подключаются модули Ma и Mb.

Рассмотрим различные типы драйверов:

- драйвер A — вызывает подчиненный модуль;
- драйвер B — посылает элемент данных (параметр) из внутренней таблицы;
- драйвер C —отображает параметр из подчиненного модуля;
- драйвер D — является комбинацией драйверов B и C.

Очевидно, что драйвер A наиболее прост, а драйвер D наиболее сложен в

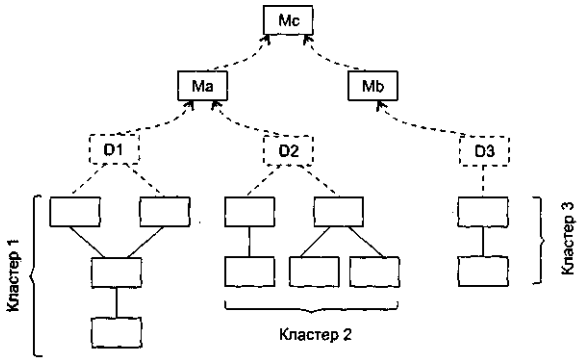
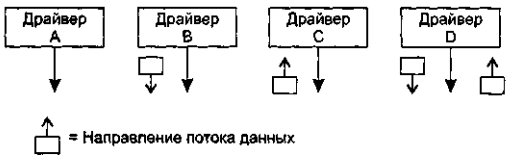


Рис. 8.6. Восходящая интеграция системы



реализации. Различные типы драйверов представлены на рис. 8.7.

Рис. 8.7. Различные типы драйверов

По мере продвижения интеграции вверх необходимость в выделении драйверов уменьшается. Как правило, в двухуровневой структуре драйверы не нужны.

Сравнение нисходящего и восходящего тестирования интеграции

Нисходящее тестирование:

1) *основной недостаток*— необходимость заглушек и связанные с ними трудности тестирования;
2) *основное достоинство* — возможность раннего тестирования главных управляющих функций.

Восходящее тестирование:

1) *основной недостаток* — система не существует как объект до тех пор, пока не будет добавлен последний модуль;
2) *основное достоинство* — упрощается разработка тестовых вариантов, отсутствуют заглушки.

Возможен комбинированный подход. В нем для верхних уровней иерархии применяют нисходящую стратегию, а для нижних уровней — восходящую стратегию тестирования [3], [13].

При проведении тестирования интеграции очень важно выявить критические модули. Признаки критического модуля:

1) реализует несколько требований к программной системе;
2) имеет высокий уровень управления (находится достаточно высоко в программной структуре);
3) имеет высокую сложность или склонность к ошибкам (как индикатор может использоваться цикломатическая сложность — ее верхний разумный предел составляет 10);
4) имеет определенные требования к производительности обработки.

Критические модули должны тестироваться как можно раньше. Кроме того, к ним должно применяться регрессионное тестирование (повторение уже выполненных тестов в полном или частичном объеме).

Тестирование правильности

После окончания тестирования интеграции программная система собрана в единый корпус, интерфейсные ошибки обнаружены и откорректированы. Теперь начинается последний шаг программного тестирования — *тестирование правильности.* Цель — подтвердить, что функции, описанные в спецификации требований к ПС, соответствуют ожиданиям заказчика [64], [69].

Подтверждение правильности ПС выполняется с помощью тестов «черного ящика», демонстрирующих соответствие требованиям. При обнаружении отклонений от спецификации требований создается список недостатков. Как правило, отклонения и ошибки, выявленные при подтверждении правильности, требуют изменения сроков разработки продукта.

Важным элементом подтверждения правильности является проверка конфигурации ПС. Конфигурацией ПС называют совокупность всех элементов информации, вырабатываемых в процессе конструирования ПС. Минимальная конфигурация ПС включает следующие базовые элементы:

1) системную спецификация;
2) план программного проекта;
3) спецификацию требований к ПС; работающий или бумажный макет;
4) предварительное руководство пользователя;
5) спецификация проектирования;
6) листинги исходных текстов программ;
7) план и методику тестирования; тестовые варианты и полученные результаты;
8) руководства по работе и инсталляции;
9) exe-код выполняемой программы;
10) описание базы данных;

11) руководство пользователя по настройке;

12) документы сопровождения; отчеты о проблемах ПС; запросы сопровождения; отчеты о конструкторских изменениях;

13) стандарты и методики конструирования ПС.

Проверка конфигурации гарантирует, что все элементы конфигурации ПС правильно разработаны, учтены и достаточно детализированы для поддержки этапа сопровождения в жизненном цикле ПС.

Разработчик не может предугадать, как заказчик будет реально использовать ПС. Для обнаружения ошибок, которые способен найти только конечный пользователь, используют процесс, включающий альфа- и бета-тестирование.

Альфа-тестирование проводится заказчиком в организации разработчика. Разработчик фиксирует все выявленные заказчиком ошибки и проблемы использования.

Бета-тестирование проводится конечным пользователем в организации заказчика. Разработчик в этом процессе участия не принимает. Фактически, бета-тестирование — это реальное применение ПС в среде, которая не управляется разработчиком. Заказчик сам записывает все обнаруженные проблемы и сообщает о них разработчику. Бета-тестирование проводится в течение фиксированного срока (около года). По результатам выявленных проблем разработчик изменяет ПС и тем самым подготавливает продукт полностью на базе заказчика.

Системное тестирование

Системное тестирование подразумевает выход за рамки области действия программного проекта и проводится не только программным разработчиком. Классическая проблема системного тестирования — указание причины. Она возникает, когда разработчик одного системного элемента обвиняет разработчика другого элемента в причине возникновения дефекта. Для защиты от подобного обвинения разработчик программного элемента должен:

1) предусмотреть средства обработки ошибки, которые тестируют все вводы информации от других элементов системы;

2) провести тесты, моделирующие неудачные данные или другие потенциальные ошибки интерфейса ПС;

3) записать результаты тестов, чтобы использовать их как доказательство невиновности в случае «указания причины»;

4) принять участие в планировании и проектировании системных тестов, чтобы гарантировать адекватное тестирование

ПС.

В конечном счете системные тесты должны проверять, что все системные элементы правильно объединены и выполняют назначенные функции. Рассмотрим основные типы системных тестов [13], [52].

Тестирование восстановления

Многие компьютерные системы должны восстанавливаться после отказов и возобновлять обработку в пределах заданного времени. В некоторых случаях система должна быть отказоустойчивой, то есть отказы обработки не должны быть причиной прекращения работы системы. В других случаях системный отказ должен быть устранен в пределах заданного кванта времени, иначе заказчику наносится серьезный экономический ущерб.

Тестирование восстановления использует самые разные пути для того, чтобы заставить ПС отказать, и проверяет полноту выполненного восстановления. При автоматическом восстановлении оцениваются правильность повторной инициализации, механизмы копирования контрольных точек, восстановление данных, перезапуск. При ручном восстановлении оценивается, находится ли среднее время восстановления в допустимых пределах.

Тестирование безопасности

Компьютерные системы очень часто являются мишенью незаконного проникновения. Под проникновением понимается широкий диапазон действий: попытки хакеров проникнуть в систему из спортивного интереса, месть рассерженных служащих, взлом мошенниками для незаконной наживы.

Тестирование безопасности проверяет фактическую реакцию защитных механизмов, встроенных в систему, на проникновение.

В ходе тестирования безопасности испытатель играет роль взломщика. Ему разрешено все:

- попытки узнать пароль с помощью внешних средств;
- атака системы с помощью специальных утилит, анализирующих защиты;
- подавление, ошеломление системы (в надежде, что она откажется обслуживать других клиентов);
- целенаправленное введение ошибок в надежде проникнуть в систему в ходе восстановления;
- просмотр несекретных данных в надежде найти ключ для входа в систему.

Конечно, при неограниченном времени и ресурсах хорошее тестирование безопасности взломает любую систему. Задача проектировщика системы — сделать цену проникновения более высокой, чем цена получаемой в результате информации.

Стрессовое тестирование

На предыдущих шагах тестирования способы «белого» и «черного ящиков» обеспечивали полную оценку нормальных программных функций и качества функционирования. Стрессовые тесты проектируются для навязывания программам ненормальных ситуаций. В сущности, проектировщик стрессового теста спрашивает, как

сильно можно расшатать систему, прежде чем она откажет?

Стрессовое тестирование производится при ненормальных запросах на ресурсы системы (по количеству, частоте, размеру-объему).

Примеры:

- генерируется 10 прерываний в секунду (при средней частоте 1,2 прерывания в секунду);
- скорость ввода данных увеличивается прямо пропорционально их важности (чтобы определить реакцию входных функций);
- формируются варианты, требующие максимума памяти и других ресурсов;
- генерируются варианты, вызывающие переполнение виртуальной памяти;
- проектируются варианты, вызывающие чрезмерный поиск данных на диске.

По существу, испытатель пытается разрушить систему. Разновидность стрессового тестирования называется *тестированием чувствительности.* В некоторых ситуациях (обычно в математических алгоритмах) очень малый диапазон данных, содержащийся в границах правильных данных системы, может вызвать ошибочную обработку или резкое понижение производительности. Тестирование чувствительности обнаруживает комбинации данных, которые могут вызвать нестабильность или неправильность обработки.

Тестирование производительности

В системах реального времени и встроенных системах недопустимо ПО, которое реализует требуемые функции, но не соответствует требованиям производительности.

Тестирование производительности проверяет скорость работы ПО в компьютерной системе. Производительность тестируется на всех шагах процесса тестирования. Даже на уровне элемента при проведении тестов «белого ящика» может оцениваться производительность индивидуального модуля. Тем не менее, пока все системные элементы не объединятся полностью, не может быть установлена истинная производительность системы. Иногда тестирование производительности сочетают со стрессовым тестированием. При этом нередко требуется специальный аппаратный и программный инструментарий. Например, часто требуется точное измерение используемого ресурса (процессорного цикла и т. д.). Внешний инструментарий регулярно отслеживает интервалы выполнения, регистрирует события (например, прерывания) и машинные состояния. С помощью инструментария испытатель может обнаружить состояния, которые приводят к деградации и возможным отказам системы.

Метрики качества программного обеспечения

На протяжении многих лет отдельные авторы и целые организации определяли термин “качество” по-разному. Фил Кросби в 1979 году дал определение качеству как “соответствие пользовательским требованиям”. Уотс Хемпфри описывает качество как “достижение отличного уровня пригодности к использованию”. Компания IBM, в свою очередь, ввела в оборот фразу “качество, управляемое рыночными потребностями”. Критерий Бэлдриджа для организационного качества использует похожую фразу - “качество, задаваемое потребителем”, рассматривая удовлетворение потребителя в качестве главного соображения в отношении качества. Чаще понятие качества используется в соответствии с определением системы менеджмента качества ИСО 9001 как “степень соответствия присущих характеристик требованиям”.

Для любого инженерного продукта существует множество интерпретаций качества, в зависимости от конкретной “системы координат”.

Сейчас существует несколько определений качества, которые в целом совместимы друг с другом.

Определение ISO: Качество - это полнота свойств и характеристик продукта, процесса или услуги, которые обеспечивают способность удовлетворять заявленным или подразумеваемым потребностям.

Определение IEEE: Качество программного обеспечения - это степень, в которой оно обладает требуемой комбинацией свойств.

Качество ПО - это относительное понятие, которое имеет смысл только при учете реальных условий его применения, поэтому требования, предъявляемые к качеству, ставятся в соответствии с условиями и конкретной областью их применения. Оно характеризуется тремя аспектами: качество программного продукта, качество процессов ЖЦ и качество сопровождения или внедрения.

Качество продукта достигается процедурами контроля промежуточных продуктов на процессах ЖЦ, проверкой их на достижение необходимого качества, а также методами сопровождения продукта. Эффект от внедрения ПС в значительной степени зависит от знаний обслуживающего персонала функций продукта и правил их выполнения.

Так, качество ПО рассматривается как очень важный аспект для разработчиков, пользователей и руководителей проектов. Качество программного обеспечения - величина, отражающая, в каком объеме в программный продукт включен набор желаемых функций для повышения эффективности ПП в течение жизненного цикла.

Для любой системы, использующей программное обеспечение, должны быть разработаны три вида спецификаций, такие как функциональные требования, требования к качеству, требования к ресурсам.

Качество включает в себя все характеристики и существенные особенности продукта или его функционирования, которые относятся к удовлетворению заданных спецификациями требований. определения качества ПО.

При определении требований к ПО задаются соответствующие им внешние характеристики и их атрибуты (подхарактеристики), определяющие разные стороны управления продуктом в заданной среде. Для набора характеристик качества ПО, приведенных в требованиях, определяются соответствующие метрики, модели их оценки и диапазон значений мер для измерения отдельных атрибутов качества.

В настоящее время в программной инженерии еще не сформировалась окончательно система метрик. Действуют разные подходы к определению их набора и методов измерения.

Система измерения включает метрики и модели измерений, которые используются для количественной оценки качества ПО.

Метрики определяются по модели измерения атрибутов ПО на всех этапах ЖЦ (промежуточная, внутренняя метрика) и особенно на этапе тестирования или функционирования (внешние метрики) продукта.

Метрика качества программ - система измерений качества программ. Эти измерения могут проводиться на уровне критериев качества программ или на уровне отдельных характеристик качества.

В исследовании метрик ПО различают два основных направления:

1) поиск метрик, характеризующих наиболее специфические свойства программ, т.е. метрик оценки самого ПО;

2) использование метрик для оценки технических характеристик и факторов разработки программ, т.е. метрик оценки условий разработки программ.

Типы метрик

По виду информации, получаемой при оценке качества ПО метрики можно разбить на три группы:

1) метрики, оценивающие отклонение от нормы характеристик исходных проектных материалов. Они устанавливают полноту заданных технических характеристик исходного кода,

2) метрики, позволяющие прогнозировать качество разрабатываемого ПО. Они заданы на множестве возможных вариантов решений поставленной задачи и их реализации и определяют качество ПО, которое будет достигнуто в итоге,

3) метрики, по которым принимается решение о соответствии конечного ПО заданным требованиям. Они позволяют оценить соответствие разработки заданным требованиям.

Типы метрик в зависимости от назначения:

- метрики программного продукта, которые используются при измерении его характеристик - свойств;
- метрики процесса, которые используются при измерении свойства процесса ЖЦ создания продукта;
- метрики использования.

Метрики программного продукта включают:

- внешние метрики, обозначающие свойства продукта, видимые пользователю;
- внутренние метрики, обозначающие свойства, видимые только команде разработчиков.

Внешние метрики продукта - это метрики:

- надежности продукта, которые служат для определения числа дефектов;
- функциональности, с помощью которых устанавливаются наличие и правильность реализации функций в продукте;
- сопровождения, с помощью которых измеряются ресурсы продукта (скорость, память, среда);
- применимости продукта, которые способствуют определению степени доступности для изучения и использования;
- стоимости, которыми определяется стоимость созданного продукта.

Внутренние метрики продукта включают:

- метрики размера, необходимые для измерения продукта с помощью его внутренних характеристик;
- метрики сложности, необходимые для определения сложности продукта;
- метрики стиля, которые служат для определения подходов и технологий создания отдельных компонентов продукта и его документов.

Внутренние метрики позволяют определить производительность продукта и являются релевантными по отношению к внешним метрикам.

Внешние и внутренние метрики задаются на этапе формирования требований к ПО и являются предметом планирования и управления достижением качества конечного программного продукта.

Метрики продукта часто описываются комплексом моделей для установки различных свойств, значений модели качества или прогнозирования. Измерения проводятся, как правило, после калибровки метрик на ранних этапах проекта. Общая мера - степень трассируемости, которая определяется числом трасс, прослеживаемых с помощью моделей сценариев типа UML и оценкой количества:

- требований;
- сценариев и действующих лиц;
- объектов, включенных в сценарий, и локализация требований к каждому сценарию;
- параметров и операций объекта и др.

Стандарт ISO/IEC 9126-2 определяет следующие типы мер:

- мера размера ПО в разных единицах измерения (число функций, строк в программе, размер дисковой памяти и др.);
- мера времени (функционирования системы, выполнения компонента и др.);
- мера усилий (производительность труда, трудоемкость и др.);
- мера учета (количество ошибок, число отказов, ответов системы и др.).

Специальной мерой может служить уровень использования повторных компонентов и измеряется как отношение размера продукта, изготовленного из готовых компонентов, к размеру системы в целом. Данная мера используется также при определении стоимости и качества ПО.

Примеры метрик:

- общее число объектов и число повторно используемых;
- общее число операций, повторно используемых и новых операций;
- число классов, наследующих специфические операции;

- число классов, от которых зависит данный класс;
- число пользователей класса или операций и др.

При оценке общего количества некоторых величин часто используются среднестатистические метрики (среднее число операций в классе, наследников класса или операций класса и др.).

Как правило, меры в значительной степени являются субъективными и зависят от знаний экспертов, производящих количественные оценки атрибутов компонентов программного продукта.

Примером широко используемых внешних метрик программ являются метрики Холстеда - это характеристики программ, выявляемые на основе статической структуры программы на конкретном языке программирования: число вхождений наиболее часто встречающихся операндов и операторов; длина описания программы как сумма числа вхождений всех операндов и операторов и др.

На основе этих атрибутов можно вычислить время программирования, уровень программы (структурированность и качество) и языка программирования (абстракции средств языка и ориентация на проблему) и др.

В качестве метрик процесса могут быть время разработки, число ошибок, найденных на этапе тестирования и др.

Практически используются следующие метрики процесса:

- общее время разработки и отдельно время для каждой стадии;
- время модификации моделей;
- время выполнения работ на процессе;
- число найденных ошибок при инспектировании;
- стоимость проверки качества;
- стоимость процесса разработки.

Метрики использования служат для измерения степени удовлетворения потребностей пользователя при решении его задач. Они помогают оценить не свойства самой программы, а результаты ее эксплуатации - эксплуатационное качество. Примером может служить - точность и полнота реализации задач пользователя, а также затраченные ресурсы (трудозатраты, производительность и др.) на эффективное решение задач пользователя.

Оценка требований пользователя проводится с помощью внешних метрик.

Стандартная оценка значений показателей качества

Оценка качества ПО согласно четырехуровневой модели качества начинается с нижнего уровня иерархии, т.е. с самого элементарного свойства оцениваемого атрибута показателя качества согласно установленных мер. На этапе проектирования устанавливают значения оценочных элементов для каждого атрибута показателя анализируемого ПО, включенного в требования.

По определению стандарта ISO/IES 9126-2 метрика качества ПО представляет собой "модель измерения атрибута, связываемого с показателем его качества".

Метрики качества используются при оценке степени тестируемости с помощью данных (*безотказная* работа, выполнимость функций, удобство применения интерфейсов пользователей, БД и т.п.) после проведения испытаний ПО на множестве тестов.

Наработка на отказ как атрибут надежности определяет среднее время между появлением угроз, нарушающих безопасность, и обеспечивает трудноизмеримую оценку ущерба, которая наносится соответствующими угрозами.

Очень часто оценка программы проводится по числу строк. При сопоставлении двух программ, реализующих одну прикладную задачу, предпочтение отдается короткой программе, так как её создает более квалифицированный персонал и в ней меньше скрытых ошибок и легче модифицировать. По стоимости она дороже, хотя времени на отладку и модификацию уходит больше. Т.е. длину программы можно использовать в качестве вспомогательного свойства для сравнения программ с учетом одинаковой квалификации разработчиков, единого стиля разработки и общей среды.

Если в требованиях к ПО было указано получить несколько показателей, то просчитанный после сбора данных показатель умножается на соответствующий весовой коэффициент, а затем суммируются все показатели для получения комплексной оценки уровня качества ПО.

На основе измерения количественных характеристик и проведения экспертизы качественных показателей с применением весовых коэффициентов, нивелирующих разные показатели, вычисляется итоговая оценка качества продукта путем суммирования результатов по отдельным показателям и сравнения их с эталонными показателями ПО (стоимость, время, ресурсы и др.).

В конечном итоге результат оценки качества являются критерием эффективности и целесообразности применения методов проектирования, инструментальных средств и методик оценивания результатов создания программного продукта на стадиях ЖЦ.

Для оценки значений показателей качества используется стандарт, в котором представлены следующие методы: измерительный, регистрационный, расчетный и экспертный (а также комбинации этих методов).

Измерительный метод базируется на использовании измерительных и специальных программных средств для получения информации о характеристиках ПО, например, определение объема, числа строк кода, операторов, количества ветвей в программе, число точек входа (выхода), реактивность и др.

Регистрационный метод используется при подсчете времени, числа сбоев или отказов, начала и конца работы ПО в процессе его выполнения.

Расчетный метод базируется на статистических данных, собранных при проведении испытаний, эксплуатации и сопровождении ПО. Расчетными методами оцениваются *показатели надежности*, точности, устойчивости, реактивности и др.

Экспертный метод осуществляется группой экспертов - специалистов, компетентных в решении данной задачи или типа ПО. Их оценка базируется на опыте и интуиции, а не на непосредственных результатах расчетов или экспериментов.

Этот метод проводится путем просмотра программ, кодов, сопроводительных документов и способствует качественной оценки созданного продукта. Для этого устанавливаются контролируемые признаки, которые коррелированы с одним или несколькими показателями качества и включены в опросные карты экспертов.

Метод применяется при оценке таких показателей, как анализируемость, документируемость, структурированность ПО и др.

Для правильного использования результатов измерений каждая мера идентифицируется шкалой измерений.

Стандарт ISO/IES 9126-2 рекомендует применять 5 видов шкал измерения значений, которые упорядочены от менее строгой к более строгой:

- номинальная шкала отражает категории свойств оцениваемого объекта без их упорядочения;
- порядковая шкала служит для упорядочивания характеристики по возрастанию или убыванию путем сравнения их с базовыми значениями;
- интервальная шкала задает существенные свойства объекта (например, календарная дата);
- относительная шкала задает некоторое значение относительно выбранной единицы;
- абсолютная шкала указывает на фактическое значение величины.

Показатели, которые вычисляются с помощью метрических шкал, называются количественные, а определяемые с помощью порядковых и классификационных шкал - качественные.

При оценке общего количества некоторых величин часто используются среднестатистические метрики (среднее число операций в классе, наследников класса или операций класса и др.).

Метрики сложности программ

Теория сложности программ ориентирована на управление качеством ПО и контроль ее эталонной сложности в период эксплуатации. В настоящее время многообразие показателей (в той или иной степени описывающих сложность программ) столь велико, что для их употребления требуется предварительное упорядочение.

В ряде случаев удовлетворяются тремя категориями метрик сложности.

Первая категория определяется как словарная метрика, основанная на метрических соотношениях Холстеда, цикломатических мерах Мак-Кейба и измерениях Тейера.

Вторая категория ориентирована на метрики связей, отражающих сложность отношений между компонентами системы - это метрики Уина и Винчестера.

Третья категория включает семантические метрики, связанные с архитектурным построением программ и их оформлением.

Согласно другой классификации, показатели сложности делятся на две группы: сложность проектирования и сложность функционирования.

Сложность проектирования, которая определяется размерами программы, количеством обрабатываемых переменных, трудоемкостью и длительностью разработки и др. факторами, анализируется на основе трех базовых компонентов: сложность структуры программы, сложность преобразований (алгоритмов), сложность данных.

Во вторую группу показателей отнесены временная, программная и информационная сложности, характеризующие эксплуатационные качества ПО.

При оценке сложности программ, как правило, выделяют три основные группы метрик:

1) метрики размера программ,
2) метрики сложности потока управления программ,
3) метрики сложности потока данных программ.

Метрики первой группы базируются на определении количественных характеристик, связанных с размером программы, и отличаются относительной простотой. К наиболее известным метрикам данной группы относятся число операторов программы, количество строк исходного текста. Метрики этой группы ориентированы на анализ исходного текста программ. Поэтому они могут использоваться для оценки сложности промежуточных продуктов разработки.

Метрики второй группы базируются на анализе управляющего графа программы. Представителем данной группы является метрика Мак-Кейба. Управляющий граф программы, который используют метрики данной группы, может быть построен на основе алгоритмов модулей. Поэтому метрики второй группы могут применяться для оценки сложности промежуточных продуктов разработки.

Метрики третьей группы базируются на оценке использования, конфигурации и размещения данных в программе. В первую очередь это касается глобальных переменных. К данной группе относятся метрики Чепина.

В настоящий момент существует весьма обширное число показателей, с помощью которых можно измерять множество различных аспектов создания программного обеспечения. Зачастую речь идет не о том, что одна метрика лучше другой. Все они позволяют посмотреть на один и тот же процесс под разными углами зрения, поэтому используются в комплексе и только так могут служить отправной точкой для принятия объективных решений.